1 Planning and Learning

1.1 Introduction

The Bellman equation appears throughout dynamic economics: consumption-savings problems, job search, investment timing, and structural estimation of discrete choice models. Standard treatments in Stokey et al. (1989) and Rust (1987) assume the transition probabilities and reward functions are known; the computational task is to find the fixed point of the Bellman operator. Reinforcement learning solves the same fixed-point problem but under two additional constraints: the transition model may be unknown (requiring learning from sampled transitions), and the state space may be too large for exact computation (requiring function approximation). This chapter develops the theoretical foundations that govern when and why RL algorithms converge.

The central insight is that RL algorithms are not mysterious: they are asymptotic approximations to classical dynamic programming operators. Value iteration becomes Q-learning when expectations are replaced by single samples. Policy iteration becomes the natural policy gradient when the greedy improvement step is approximated by gradient ascent. The stochastic approximation framework of Robbins (1952) provides the mathematical foundation: under appropriate step-size conditions, noisy iterates converge to the same fixed points as their deterministic counterparts.

1.2 The Newton Connection

Value iteration (VI) and policy iteration (PI) are the workhorses of dynamic programming (Stokey et al., 1989). VI applies the Bellman operator repeatedly until convergence; PI alternates between policy evaluation (solving a linear system) and policy improvement (taking the greedy action). PI converges faster. Why? The answer reveals a connection between dynamic programming and numerical optimization: policy iteration is Newton's method applied to the Bellman equation.

Consider the Bellman optimality operator T acting on value functions:

$$(TV)(s) = \max_{a \in \mathcal{A}} \left\{ r(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a) V(s') \right\}. \tag{1}$$

This operator is nonlinear due to the max. Value iteration applies T repeatedly: $V_{k+1} =$

TV_k. Since T is a γ-contraction in the supremum norm, Banach's fixed-point theorem guarantees $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$. This is Picard iteration, with linear convergence at rate γ.

Policy iteration takes a different approach. At the current value estimate $\tilde{V}$, define the greedy policy $\tilde{\pi}(s) = argmax_a \{r(s,a) + \gamma \sum_{s'} P(s'|s,a)\tilde{V}(s')\}$. The policy evaluation step solves the linear fixed-point equation $V = T^{\tilde{\pi}} V$ exactly, where $T^{\tilde{\pi}}$ is the policy-specific Bellman operator:

$$(T^{\tilde{\pi}} V)(s) = r(s, \tilde{\pi}(s)) + \gamma \sum_{s'} P(s'|s, \tilde{\pi}(s)) V(s'). \tag{2}$$

The key insight, formalized by Puterman and Brumelle (1979), is geometric: the linear operator $T^{\tilde{\pi}}$ is a supporting hyperplane to the nonlinear operator T at the current iterate. Specifically, the operators satisfy tangency ($T^{\tilde{\pi}} \tilde{V} = T\tilde{V}$, agreeing at the current point) and support ($T^{\tilde{\pi}} V \leq TV$ for all V, with the linear operator lying below the nonlinear one everywhere). Policy evaluation solves for the fixed point of this linearization exactly. This is precisely the structure of Newton's method: linearize the nonlinear equation at the current point, solve the linearized system, and iterate.[1]

The consequence is quadratic convergence near the optimum. While VI requires $k = \log(100)/\log(1/\gamma)$ iterations to reduce error by a factor of 100, PI typically converges in 5–10 iterations regardless of γ. Table 1 illustrates the difference.

Table 1: Iterations for 100× error reduction: VI vs. PI

γ	Value Iteration	Policy Iteration
0.90	44	5–8
0.95	90	5–8
0.99	459	5–10

Bertsekas (2022) extends this interpretation to a broad class of dynamic programming problems. The Newton structure applies whenever the Bellman operator can be written as a pointwise maximum over linear operators: $T = \max_\pi T^\pi$. This includes not only discounted MDPs but also stochastic shortest path problems, average-cost MDPs, and model predictive control formulations. The practical implication is that algorithms with policy-improvement structure (evaluate a policy exactly or approximately, then improve)

[1] The Newton interpretation of policy iteration has precursors in Kleinman (1968) for Riccati equations in linear-quadratic control and Pollatschek and Avi-Itzhak (1969) for stochastic games.

inherit Newton-like convergence behavior, while pure value-iteration methods (apply T directly) are limited to linear convergence.[2]

1.3 Stochastic Approximation

The Bellman operator in Equation (1) requires computing an expectation: $\mathbb{E}_{s' \sim P(\cdot|s,a)}[V(s')]$. In structural estimation, this typically means either knowing the transition matrix P explicitly or integrating over a high-dimensional distribution. But what if we only observe sampled transitions (s, a, r, s') from an unknown environment? Can a single noisy sample replace the expectation and still yield convergence?

The answer is yes, and the mathematical foundation is stochastic approximation, developed by Robbins (1952). Consider the problem of finding x^* such that $g(x^*) = 0$, where we cannot evaluate g directly but can observe noisy samples $g(x)+\epsilon$. The Robbins-Monro iteration is:

$$x_{t+1} = x_t - \alpha_t[g(x_t) + \epsilon_t], \tag{3}$$

where ϵ_t is zero-mean noise. Under two conditions on the step sizes, this converges to x^* with probability one: $\sum_{t=0}^{\infty} \alpha_t = \infty$ (sufficient exploration, ensuring we never stop learning) and $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$ (diminishing noise, ensuring the variance of cumulative updates is finite). The canonical choice $\alpha_t = 1/(t+1)$ satisfies both conditions.

Q-learning (Watkins and Dayan, 1992) is Robbins-Monro applied to the Bellman equation for action-value functions. Define the Q-factor Bellman operator:

$$(FQ)(s, a) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q(s', a'). \tag{4}$$

The Q-learning update, upon observing transition (s_t, a_t, r_t, s_{t+1}), is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]. \tag{5}$$

The term in brackets is a noisy sample of $(FQ)(s_t, a_t) - Q(s_t, a_t)$, the Bellman residual. The noise arises because we observe a single next state s_{t+1} rather than the full distribution $P(\cdot|s_t, a_t)$.

[2]For discounted MDPs with finite state and action spaces, there exists a stationary deterministic policy that is optimal for all initial states simultaneously (Blackwell, 1962). This Blackwell optimality guarantees that PI and VI converge to the same policy regardless of initialization.

Theorem 1 (Q-Learning Convergence)**.** *Under the conditions: (i) every state-action pair (s, a) is visited infinitely often, (ii) step sizes satisfy the Robbins-Monro conditions $\sum_t \alpha_t(s, a) = \infty$ and $\sum_t \alpha_t(s, a)^2 < \infty$ for each (s, a), Q-learning converges to Q^* with probability one.*

This result, established by Watkins and Dayan (1992) and Jaakkola et al. (1994), shows that model-free learning is possible: we can find the optimal value function without ever estimating the transition probabilities.[3]

SARSA provides an on-policy variant. Instead of taking the maximum over next actions, SARSA uses the action actually taken:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]. \tag{6}$$

This solves for the value function of the behavior policy π rather than the optimal policy. Singh et al. (2000) prove convergence under the same step-size conditions, provided the behavior policy converges to a stationary distribution.

Q-learning is noisy value iteration on Q-factors; SARSA is noisy policy evaluation. The Robbins-Monro conditions ensure that the noise averages out faster than the signal decays, allowing asymptotic convergence despite using only single-sample estimates.

Rollout methods extend this framework by performing one step of policy improvement from a base policy. AlphaZero (Silver et al., 2018) implements this principle at scale: neural networks provide a base value estimate V^μ, and Monte Carlo Tree Search extends the effective lookahead. Given base policy μ, the rollout policy selects:

$$\tilde{\pi}(s) = \underset{a}{argmax} \left\{ r(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\mu(s') \right\}. \tag{7}$$

Bertsekas (2021) proves the cost improvement property: $V^{\tilde{\pi}}(s) \leq V^\mu(s)$ for all s, with strict inequality unless μ is already optimal. Each MCTS iteration is a sampled rollout, and the aggregation across many samples approximates the expectation in the Bellman operator.[4]

[3] Model-free methods are essential when: (a) the environment is a physical system where a simulator exists but the underlying dynamics are too complex to write down (robotics, physics engines), or (b) the agent learns directly from interaction without access to a model (online learning, real-world deployment).

[4] The error bound for H-step lookahead with terminal approximation V^μ is $\|V^{\tilde{\pi}} - V^*\|_\infty \leq \frac{2\gamma^H}{1-\gamma} \|V^\mu - V^*\|_\infty$. At $H = 50$ and $\gamma = 0.99$, this multiplier is approximately 1.2; longer lookahead compensates for worse value approximations.

1.4 The Deadly Triad

The convergence results of the previous section assume tabular representations: a separate value for each state-action pair. When the state space is large or continuous, this is infeasible. The standard remedy is function approximation: represent $V(s) \approx \hat{V}(s;\theta)$ using a parameterized family such as polynomials, splines, or neural networks. Regression is then used to fit θ from sampled Bellman targets. This combination of regression and dynamic programming, however, can diverge.

Temporal difference (TD) learning with function approximation does not minimize Bellman error directly. Instead, it finds a fixed point of the projected Bellman operator ΠT^π, where Π projects onto the function approximation subspace. Consider linear function approximation $\hat{V}(s;\theta) = \phi(s)^\top\theta$, where $\phi(s) \in \mathbb{R}^d$ is a feature vector. The projection Π maps any value function to its best approximation in the span of the features, under some norm.

Tsitsiklis and Van Roy (1997) establish the convergence conditions. Define the d^π-weighted norm $\|V\|_{d^\pi}^2 = \sum_s d^\pi(s)V(s)^2$, where d^π is the stationary distribution of the Markov chain induced by policy π. When samples are drawn from d^π (on-policy sampling), the projection Π is non-expansive in this norm: $\|\Pi V_1 - \Pi V_2\|_{d^\pi} \leq \|V_1 - V_2\|_{d^\pi}$. Since T^π is a γ-contraction in any weighted ℓ_∞ norm, the composition ΠT^π remains a contraction, and TD converges to the unique fixed point.

Off-policy sampling breaks this argument; Tsitsiklis and Van Roy (1997) prove that the projected Bellman operator ΠT^π is a contraction only when Π is computed under the stationary distribution d^π. Baird (1995) constructs a counterexample: a six-state MDP with two-dimensional linear function approximation, where samples are drawn uniformly rather than from d^π. The TD updates cause the weight vector to diverge to infinity. The geometric intuition is that uniform sampling induces a different norm under which Π is no longer non-expansive; the composition ΠT^π ceases to be a contraction and can amplify errors.

This phenomenon is called the deadly triad: the combination of function approximation, bootstrapping (TD targets that depend on current estimates), and off-policy learning. Removing any one element restores convergence. Without function approximation (tabular), the projection Π is the identity and convergence follows from Section 1.3.

Without bootstrapping (Monte Carlo returns), targets are independent of current estimates and regression is well-posed. Without off-policy learning (on-policy sampling), the Tsitsiklis-Van Roy conditions are satisfied.

DQN (Mnih et al., 2015) addresses the deadly triad through two mechanisms. Experience replay stores transitions in a buffer and samples mini-batches uniformly, breaking temporal correlation and making the sampling distribution approximately stationary. Target networks hold the regression target fixed for many updates: instead of regressing toward $r + \gamma \max_{a'} Q(s', a'; \theta)$, DQN regresses toward $r + \gamma \max_{a'} Q(s', a'; \theta^-)$, where θ^- is updated slowly. This decouples the moving target problem from the parameter updates.[5]

1.5 Policy Gradient

Value-based methods find fixed points of the Bellman operator. Policy-based methods take a different approach: parameterize the policy directly as $\pi_\theta(a|s)$ and maximize expected return $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_{t=0}^{\infty} \gamma^t R_t]$ by gradient ascent. This formulation sidesteps the Bellman equation entirely and frames reinforcement learning as constrained optimization.

The policy gradient theorem (Sutton et al., 2000) provides a tractable expression for the gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^{\pi_\theta}, a \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) \, Q^{\pi_\theta}(s, a) \right], \tag{8}$$

where $d^{\pi_\theta}(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s | \pi_\theta)$ is the discounted state visitation distribution. The gradient does not require $\nabla_\theta d^{\pi_\theta}$, the derivative of the state distribution with respect to policy parameters. This term is intractable in general, but it cancels in the derivation. The gradient depends only on quantities computable from sampled trajectories: the policy π_θ, its gradient $\nabla_\theta \log \pi_\theta$, and the action-value Q^{π_θ}. The transition dynamics $P(s'|s, a)$ do not appear, making the gradient estimable without a model of the environment.

REINFORCE (Williams, 1992) is the simplest policy gradient algorithm. Sample a trajectory $(s_0, a_0, r_0, s_1, \ldots)$, compute the return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ from each time step, and update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \, G_t. \tag{9}$$

This is an unbiased estimator of $\nabla_\theta J(\theta)$, but its variance is high because a single trajectory

[5] Sutton et al. (2009) reformulate TD as a saddle-point problem, yielding gradient TD methods that converge off-policy with linear function approximation. These methods are less commonly used in practice than target networks.

provides a noisy estimate of Q^{π_θ}.

Standard gradient descent treats all parameter directions equally. But small changes in θ can cause large changes in the policy distribution π_θ. The natural policy gradient (Kakade, 2001) accounts for this curvature by preconditioning with the Fisher information matrix:

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1}\nabla_\theta J(\theta), \quad F(\theta) = \mathbb{E}_{s,a}\left[\nabla_\theta \log \pi_\theta(a|s)\nabla_\theta \log \pi_\theta(a|s)^\top\right]. \tag{10}$$

In the tabular setting, NPG is equivalent to policy iteration: the natural gradient direction exactly matches the policy improvement step. This explains why NPG inherits the fast convergence of PI.

The RL objective $J(\theta)$ is non-convex, raising concerns about local optima. Agarwal et al. (2021) establish that for softmax policies, $J(\theta)$ satisfies a gradient domination condition:

$$\|\nabla_\theta J(\theta)\|_2^2 \geq \frac{(1-\gamma)^3}{8}\left(J(\pi^*) - J(\pi_\theta)\right). \tag{11}$$

This Polyak-Łojasiewicz-type inequality implies that every stationary point is globally optimal. NPG with constant step size $\eta = (1-\gamma)/4$ achieves $J(\pi^*) - J(\pi_k) \leq O(1/k)$, with no dependence on $|\mathcal{S}|$ or $|\mathcal{A}|$.

The performance difference lemma provides the foundation for trust region methods. For any two policies π and π':

$$J(\pi') - J(\pi) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi'}}\left[\sum_a \pi'(a|s)A^\pi(s,a)\right], \tag{12}$$

where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the advantage function. This identity underlies TRPO (Schulman et al., 2015) and PPO (Schulman et al., 2017), which constrain policy updates to regions where the advantage estimates remain accurate.

Adding entropy regularization accelerates convergence. The entropy-regularized objective is:

$$J_\tau(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\left(R_t + \tau\mathcal{H}(\pi_\theta(\cdot|s_t))\right)\right], \tag{13}$$

where $\mathcal{H}(\pi) = -\sum_a \pi(a)\log\pi(a)$ is the entropy and $\tau > 0$ is the temperature. Cen et al. (2022) prove that NPG with entropy regularization achieves linear convergence: $J_\tau(\pi^*) - J_\tau(\pi_k) \leq (1-c\tau)^k \cdot [J_\tau(\pi^*) - J_\tau(\pi_0)]$ for some constant $c > 0$. The optimal

policy under entropy regularization is softmax: $\pi^*(a|s) \propto \exp(Q^*(s,a)/\tau)$, which matches the logit choice probability from discrete choice econometrics.

1.6 Actor-Critic

REINFORCE uses the sample return G_t to estimate the policy gradient. This estimator is unbiased but has high variance: a single trajectory provides a noisy signal. TD methods use bootstrapped estimates $r + \gamma V(s')$ as targets. These have lower variance (the value function aggregates information across trajectories) but are biased when V is approximate. Actor-critic methods combine both: the critic estimates the value function, the actor updates the policy using the critic's estimates.

The theoretical foundation is two-timescale stochastic approximation (Konda and Tsitsiklis, 2000). Run two concurrent learning processes:

$$\text{Critic (fast):} \quad \theta_{t+1} = \theta_t + \alpha_t^{(c)} \delta_t \nabla_\theta \hat{V}(s_t; \theta_t), \tag{14}$$

$$\text{Actor (slow):} \quad \omega_{t+1} = \omega_t + \alpha_t^{(a)} \delta_t \nabla_\omega \log \pi_\omega(a_t|s_t), \tag{15}$$

where $\delta_t = r_t + \gamma \hat{V}(s_{t+1}; \theta_t) - \hat{V}(s_t; \theta_t)$ is the TD error. The critic updates the value function parameters θ; the actor updates the policy parameters ω.

Convergence requires the critic to learn faster than the actor:

$$\lim_{t \to \infty} \frac{\alpha_t^{(a)}}{\alpha_t^{(c)}} = 0, \quad \text{with both satisfying Robbins-Monro conditions.} \tag{16}$$

Under this separation, the actor sees a quasi-stationary critic: from the actor's perspective, the critic provides approximately correct value estimates at each step. The actor's updates are then approximately unbiased policy gradient steps. Konda and Tsitsiklis (2000) prove convergence to a local optimum of $J(\omega)$.

A2C (Advantage Actor-Critic) is the synchronous variant: collect a batch of transitions, compute TD errors, and update both networks. A3C (Mnih et al., 2016) parallelizes this across multiple workers updating a shared parameter server asynchronously. SAC (Soft Actor-Critic) (Haarnoja et al., 2018) adds entropy regularization, using the soft Bellman operator for the critic and entropy-augmented returns for the actor.

The actor-critic structure separates identification from optimization. The critic solves

a regression problem (estimate V^π from data), while the actor solves an optimization problem (improve π using the estimated values).

1.7 Bridging Results

The preceding sections establish that RL algorithms converge under appropriate conditions. Two questions remain: how does approximation error in the value function propagate to the policy, and how does computational complexity scale with the problem size?

Singh and Yee (1994) bound the policy degradation from value function errors. If $\hat{V}$ approximates V^* with error $\|\hat{V} - V^*\|_\infty \leq \epsilon$, and $\hat{\pi}$ is the greedy policy with respect to $\hat{V}$, then:

$$\|V^* - V^{\hat{\pi}}\|_\infty \leq \frac{2\gamma}{1-\gamma}\epsilon. \tag{17}$$

At $\gamma = 0.99$, the amplification factor is $2 \cdot 0.99/0.01 = 198$. A 1% error in value function approximation yields at most 198% error in policy value. This bound is pessimistic but finite: approximate value functions do not cause unbounded policy degradation.

Classical dynamic programming complexity scales with the state space size $|\mathcal{S}|$. For problems like Go, where $|\mathcal{S}| \approx 10^{170}$, exact computation is impossible. Kearns et al. (2002) prove that with access to a generative model (a simulator that samples transitions from any state-action pair), near-optimal planning requires samples polynomial in the effective horizon $H = O(\log(1/\epsilon)/\log(1/\gamma))$, the branching factor $|\mathcal{A}|$, and the desired accuracy $1/\epsilon$, with no dependence on $|\mathcal{S}|$. The sparse sampling algorithm builds a random tree of depth H from the current state, estimates values by averaging leaf rewards, and selects the action with highest estimated value. This result explains why Monte Carlo Tree Search succeeds in large state spaces: planning complexity depends on the horizon and action space, not the state space cardinality.

1.8 Conclusion

Reinforcement learning is not a departure from dynamic programming but an extension of it. The algorithms in this chapter are asymptotic approximations to the Bellman fixed-point operator, justified by the mathematics of contractions, stochastic approximation, and gradient domination. Policy iteration is Newton's method; Q-learning is noisy value

iteration; natural policy gradient recovers policy iteration in the tabular case. The theoretical framework provides guarantees: convergence rates, error bounds, and conditions under which approximation is safe.

The central challenge in reinforcement learning is overcoming the deadly triad. Methods that address it directly—through on-policy sampling (PPO), target networks (DQN), or entropy regularization (SAC)—have achieved the most significant empirical successes.

References

Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98), 2021.

Leemon Baird. Residual algorithms: Reinforcement learning with function approximation. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 30–37. Morgan Kaufmann, 1995.

Dimitri P. Bertsekas. Lessons from AlphaZero for optimal, model predictive, and adaptive control. *Athena Scientific Reports*, 2021.

Dimitri P. Bertsekas. Newton's method for reinforcement learning and model predictive control. *Results in Control and Optimization*, 7:100121, 2022.

David Blackwell. Discrete dynamic programming. *Annals of Mathematical Statistics*, 33 (2):719–726, 1962.

Shicong Cen, Chen Cheng, Yuxin Chen, Yuting Wei, and Yuejie Chi. Fast global convergence of natural policy gradient methods with entropy regularization. *Operations Research*, 70(4), 2022.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870, 2018.

Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of

stochastic iterative dynamic programming algorithms. In *Advances in Neural Information Processing Systems 6*, pages 703–710. Morgan Kaufmann, 1994.

Sham M. Kakade. A natural policy gradient. *Advances in Neural Information Processing Systems*, 14, 2001.

Michael Kearns, Yishay Mansour, and Andrew Y. Ng. A sparse sampling algorithm for near-optimal planning in large Markov decision processes. *Machine Learning*, 49(2–3): 193–208, 2002.

David L. Kleinman. On an iterative technique for Riccati equation computations. *IEEE Transactions on Automatic Control*, 13(1):114–115, 1968.

Vijay R. Konda and John N. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems 12*, pages 1008–1014. MIT Press, 2000.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1928–1937. PMLR, 2016.

Moshe A. Pollatschek and Benjamin Avi-Itzhak. Algorithms for stochastic games with geometrical interpretation. *Management Science*, 15(7):399–415, 1969.

Martin L. Puterman and Shelby L. Brumelle. On the convergence of policy iteration in stationary dynamic programming. *Mathematics of Operations Research*, 4(1):60–69, 1979.

Herbert Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952.

John Rust. Optimal replacement of gmc bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033, 1987.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. *Proceedings of the 32nd International Conference on Machine Learning*, pages 1889–1897, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419): 1140–1144, 2018.

Satinder Singh, Tommi Jaakkola, Michael L Littman, and Csaba Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine learning*, 38:287–308, 2000.

Satinder P. Singh and Richard C. Yee. An upper bound on the loss from approximate optimal-value functions. *Machine Learning*, 16(3):227–233, 1994.

Nancy L. Stokey, Robert E. Lucas, and Edward C. Prescott. *Recursive Methods in Economic Dynamics*. Harvard University Press, Cambridge, MA, 1989.

Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 2000.

Richard S. Sutton, Hamid Reza Maei, Doina Precup, Shalabh Bhatnagar, David Silver, Csaba Szepesvári, and Eric Wiewiora. Fast gradient-descent methods for temporal-difference learning with linear function approximation. In *Proceedings of the 26th International Conference on Machine Learning*, pages 993–1000. ACM, 2009.

John N. Tsitsiklis and Benjamin Van Roy. Analysis of temporal-difference learning with function approximation. In *Advances in Neural Information Processing Systems 9 (NIPS)*, 1997.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4): 279–292, 1992.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.